AI ENGINEERING JOURNAL

Team name: NEW AURA

AI Tools used: Claude (Claude Code, Anthropic): specification, solver core, API, web UI, tests and documents. Groq API with openai/gpt-oss-120b, inside the product, optional and opt-in. OpenAI Codex, by a teammate on the engineering branch that extended routing and planner workflow. Claude and Microsoft Edge text-to-speech (edge-tts, voice Emma), by a teammate who recorded the screen himself, to edit the English subtitles and produce the voice-over of MVP.mp4.

Language: English preferred

1. AI Strategy

Briefly describe how your team used AI during the project.

(Max. 150 words).

We used AI as a fast pair programmer and drafter, never as the source of numbers. Claude helped us draft the specification, scaffold the OR-Tools solver, build the FastAPI and React app, and write tests and documents. Inside the product, a Groq-hosted model is optional: it only rewrites facts that our own code has already calculated.

Every AI contribution passed a human gate: a golden test (cost 700 falls to 500), an independent plan verifier, team review of the specification, manual walk-throughs of the full journey, and a number guard that rejects any explanation containing a figure that is not in the computed facts. A planner must approve or reject each plan, and AI text stays a draft until approved. We logged where AI was wrong and fixed it in code, for example savings reported against an invalid baseline.

2. AI Usage log

Document the most significant AI interactions during your project. Maximum 5 tasks

Stage	Purpose of using AI	Original time estimate	Actual time with AI	Human Validation	Outcome

Design (2 Nov)	Explain the Round 1 idea and draft the MVP specification and 7-day plan for team review.	about 8 h	about 3 h	Teammates reviewed the draft and recorded 16 decisions to settle (D01-D16).	Specification kept; open decisions are tracked in the repo. The team set the rules that data is synthetic and AI does not calculate.
Build (2 Nov)	Scaffold the solver core: OR-Tools LP and MILP, validation, preferred-warehouse baseline, independent verifier, tests.	about 16 h	about 2.5 h	Golden test (cost 700 falls to 500), independent verifier that does not use the solver, and a written review that found 7 issues.	Rejected the first version: it reported savings against an invalid baseline and accepted unmet demand. Strict verifier added; saving is null and the UI says unusable.
Build (2-4 Nov)	Generate the FastAPI service, SQLite storage, approve and reject flow, hash-chained audit log, export and the first web UI.	about 20 h	about 4 h	More than 50 automated tests, a Playwright walk-through of the whole journey, and manual approve, reject and export checks.	Export is blocked until a plan is approved; approved plans are locked. Audit log is described as tamper-evident, not tamper-proof.
Build (4-5 Nov)	Write and tune the optional AI explanation (Groq) with guardrails.	about 8 h	about 3 h	Number guard checks every figure against computed facts; live check of 3 drafts on 6 Nov; manual review of wording.	Fixed truncated output and a false rejection of numbers written as 32 698. Reviewer found causal claims the data does not prove, so drafts stay unapproved until a planner edits them.
Design (4-5 Nov)	Redesign the web UI (React, Tailwind) using a UI/UX design skill, then remove debug-style content.	about 16 h	about 3.5 h	Checked every screen in the browser at desktop and 375 px width, and re-ran the approve	Reorganised the screens into three planning steps plus audit trail, what-if

				and export journey.	and guide; hid technical codes; kept the synthetic data label visible on every screen.

3. Key AI Decisions

Describe **2–3 examples** where AI had a meaningful impact on your project.

Task	AI suggestion	What your team did
Check the baseline used for savings.	The first generated code compared the optimised plan with a greedy baseline even when that baseline could not serve all demand, so it showed savings that were not real.	We made the verifier strict and the saving null when the baseline is unusable. The UI shows unusable, and a regression test with a greedy-trap dataset protects it.
Write the plan explanation.	The model produced fluent text that blamed Warehouse A being full for every move, although our data shows only that A and B were full and that allocations changed. Earlier it also cut off mid-sentence.	We kept AI optional and per request, gave it only code-computed facts plus route cost per pallet, rejected any figure not in the facts, and left every draft unapproved until a planner edits it. Route figures are called estimates.
Tune the synthetic demo data.	AI-built demo data first gave about 1% saving, and cost inputs were adjusted to reach about 4.2%.	We labelled all data synthetic on every screen and in the documents. We state that 4.2% is not a forecast and does not prove the Round 1 5% target. A robustness run over 11 random synthetic cases reports the whole range.

4. Reflection

Answer each question in 2-4 sentences

1. Where did AI help your team the most?

   Speed on well-defined engineering work: a working solver, API, tests and a polished web app within a few days. It also helped us turn review findings into code quickly and keep the documents consistent with the product.

2. What AI output did you reject or modify, and why?
   We rejected a baseline and verifier that hid infeasible results, an AI explanation that stated causes the data does not prove, and numbers that were not in the

computed facts. We changed the code, not just the prompt, so the checks run every time.

3. What engineering or business decisions required human judgment?
   Choosing exact optimisation over a heuristic, requiring planner approval before any plan is exported, and deciding what we may not claim: no forecast from synthetic data, no ERP integration, and no planner time or on-time delivery gain because we did not measure them.

4. Are there any obstacles when using AI?
   Reasoning models used up their token budget and cut text short, free-tier model names changed, and fluent wording can sound more certain than the evidence. We needed strict checks, honest labels and a deterministic fallback so the demo never depends on the network.